

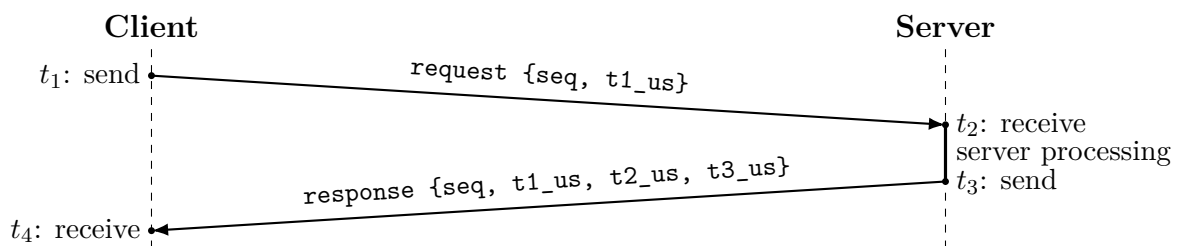
NETWORK PROGRAMMING LABORATORY

7 September 2026

Write a Rust program that estimates the clock difference between two hosts. The program uses a simplified version of the four-timestamp exchange used by NTP. Communication is unicast UDP and every datagram contains JSON. The same executable operates either as server or client.

Protocol

Each sample consists of exactly two UDP datagrams: one *request* from the client and one *response* from the server. The four timestamps identify events in this exchange; they are not four separate messages.



The request carries a sequence number and t_1 , recorded by the client just before transmission. The response copies both fields and adds t_2 , recorded when the server receives the request, and t_3 , recorded just before its reply. The client records t_4 locally after reception: t_4 is therefore *not* transmitted. All timestamps are signed microseconds since the Unix epoch. Example messages are:

```
{ "type": "request", "seq": 1, "t1_us": 1788264000000000 }
{ "type": "response", "seq": 1, "t1_us": 1788264000000000,
  "t2_us": 1788264000000410, "t3_us": 1788264000000430 }
```

The client accepts a response only if its type, sequence number, and copied t_1 match the current request. Malformed or unrelated datagrams must not cause a panic.

Command Line

```
clockprobe (-s|--server) <bind-address> [-v|--verbose]
clockprobe (-c|--client) <server-address> [-v|--verbose]
```

For example, start the two programs with:

```
clockprobe --server 0.0.0.0:5000 --verbose
clockprobe -c 192.0.2.20:5000 -v
```

Exactly one of `-s/--server` and `-c/--client` is required. The addresses are numeric IPv4 socket addresses. The optional `-v/--verbose` flag prints peer addresses and timestamps for individual exchanges. The client sends exactly 10 requests, separated by 200 ms, and waits at most one second for each reply.

Measurements

For every valid reply, compute the network delay d and clock offset o :

$$d = (t_4 - t_1) - (t_3 - t_2), \quad o = \frac{(t_2 - t_1) + (t_3 - t_4)}{2}.$$

The client and server clocks may show different values, so subtracting one server timestamp from one client timestamp does not directly give the network delay. The term $(t_3 - t_2)$ removes the time spent at the server. The two quantities $(t_2 - t_1)$ and $(t_3 - t_4)$ contain the clock offset with opposite network-delay contributions; averaging them estimates the offset when the two path delays are approximately equal. This symmetry is an assumption of the measurement, not a property guaranteed by UDP.

Interpreting the results. A positive offset means that the server clock is ahead of the client clock; a negative value means that it is behind. The delay estimates the total travel time in both directions after removing the time spent by the server between t_2 and t_3 . Different samples may produce different values because of queueing and scheduling. The reported offset is only an estimate: strong path asymmetry introduces an error that this simple protocol cannot detect.

Expected Output

The numeric values depend on the hosts and network. A possible client output is:

```
seq=1 delay=0.820 ms offset=+12.405 ms
seq=2 delay=0.730 ms offset=+12.381 ms
seq=3 timeout
...
Summary
sent: 10
received: 9
lost: 1
delay min/avg/max: 0.730/0.910/1.240 ms
average clock offset: +12.390 ms
```

Only valid replies contribute to the delay and offset statistics. Therefore, **received** is also the number of samples used to compute the reported minimum, average, and maximum.

System Time Hint

Use `SystemTime`, rather than `Instant`, because timestamps created on different hosts must refer to the same conventional origin. The following code obtains a signed timestamp in microseconds:

```
use std::time::{SystemTime, UNIX_EPOCH};

let now_us = match SystemTime::now().duration_since(UNIX_EPOCH) {
    Ok(duration) => duration.as_micros() as i64,
    Err(error) => {
        eprintln!("clock error: {error}");
        return;
    }
};
```

The Unix epoch provides a common reference for t_1 , t_2 , t_3 , and t_4 . The clocks need not agree: estimating their difference is the purpose of the exercise. **Instant** is appropriate for elapsed time inside one process, but its origin is local and its values must not be exchanged between hosts.